
Gestion de Alertas – Inventario_SE (PROYECTO_SECURITAS)

Securitas Colombia S.A.



Gestion de Alertas – Inventario_SE (PROYECTO_SECURITAS)

Guia completa de arquitectura, UX y mejores practicas para el modulo de alertas de la aplicacion de inventario.

1. Arquitectura del modulo de alertas

1.1 Estructura recomendada (FastAPI + Flet)

Ver estructura del proyecto

```
inventario_se/
+-- backend/
|   +-- alerts/
|   |   +-- models.py           # Modelo Alert en SQLAlchemy
|   |   +-- schemas.py         # Esquemas Pydantic
|   |   +-- rules.py           # Motor de reglas (evaluacion automatica)
|   |   +-- router.py          # Endpoints REST /alerts
|   |   +-- notifications.py    # Envio de notificaciones
+-- frontend/
|   +-- views/
|   |   +-- alert_dashboard.py  # Vista principal del panel
|   +-- components/
|   |   +-- alert_table.py      # Tabla de alertas con filtros
|   |   +-- alert_badge.py      # Badge de conteo (sidebar)
|   |   +-- alert_card.py       # Tarjeta de alerta individual
|   |   +-- alert_toast.py      # Notificacion flotante tipo toast
```

2. Modelo de datos

2.1 Tabla **alerts** en MySQL

Ver DDL de la tabla alerts

```
CREATE TABLE alerts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME ON UPDATE CURRENT_TIMESTAMP,  
  resolved_at DATETIME NULL,  
  tipo VARCHAR(50) NOT NULL, -- 'stock_bajo', 'vencimiento',  
  ↪ 'sin_movimiento', 'sobrestock', 'discrepancia'  
  prioridad ENUM('critica','alta','media','baja') NOT NULL,  
  estado ENUM('activa','reconocida','resuelta','ignorada') NOT  
  ↪ NULL DEFAULT 'activa',  
  titulo VARCHAR(200) NOT NULL,  
  descripcion TEXT,  
  item_id INT NOT NULL,  
  item_nombre VARCHAR(200),  
  valor_actual DECIMAL(10,2),  
  valor_umbral DECIMAL(10,2),  
  unidad VARCHAR(20),  
  asignado_a INT NULL, -- FK a users  
  notas TEXT NULL,  
  FOREIGN KEY (item_id) REFERENCES items(id),  
  INDEX idx_estado_prioridad (estado, prioridad),  
  INDEX idx_tipo (tipo),  
  INDEX idx_created_at (created_at)  
);
```

2.2 Tabla **alert_rules** (reglas configurables)

Ver DDL de la tabla alert_rules

```
CREATE TABLE alert_rules (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  tipo VARCHAR(50) NOT NULL,  
  activa BOOLEAN NOT NULL DEFAULT TRUE,  
  prioridad ENUM('critica','alta','media','baja') NOT NULL,
```

```
condicion JSON NOT NULL, -- {"campo": "cantidad",
↪ "operador": "lt", "valor": "stock_minimo"}
descripcion TEXT,
cooldown_h INT DEFAULT 24 -- Horas antes de re-disparar la
↪ misma alerta
);
```

3. Motor de reglas

Archivo: backend/alerts/rules.py

[!NOTE] El motor recorre todos los items activos y evalua cada regla. Solo crea una alerta si **no existe ya una activa o reconocida** para el mismo item y tipo.

Ver implementacion del motor de reglas

```
from datetime import datetime, timedelta
from app.db import SessionLocal
from app.items.models import Item
from app.alerts.models import Alert

REGLAS = [
    {
        "tipo": "stock_bajo",
        "prioridad": "critica",
        "titulo": "Stock critico: {nombre}",
        "condicion": lambda item: item.cantidad < item.stock_minimo,
        "descripcion": "Cantidad {actual} menor al minimo {umbral}",
    },
    {
        "tipo": "vencimiento",
        "prioridad": "alta",
        "titulo": "Proximo a vencer: {nombre}",
        "condicion": lambda item: item.fecha_vencimiento and
            item.fecha_vencimiento <= datetime.now() +
↪ timedelta(days=30),
        "descripcion": "Vence el {fecha}",
    },
    {
        "tipo": "sin_movimiento",
```

```
        "prioridad": "media",
        "titulo": "Sin movimiento: {nombre}",
        "condicion": lambda item: item.ultima_transaccion and
            item.ultima_transaccion < datetime.now() -
↪ timedelta(days=90),
        "descripcion": "Sin movimiento desde hace {dias} dias",
    },
    {
        "tipo": "sobrestock",
        "prioridad": "baja",
        "titulo": "Exceso de inventario: {nombre}",
        "condicion": lambda item: item.stock_maximo and item.cantidad >
↪ item.stock_maximo,
        "descripcion": "Cantidad {actual} supera maximo {umbral}",
    },
]

def evaluar_alertas():
    """Corre el motor de reglas. Llamar desde un scheduler periodico."""
    db = SessionLocal()
    items = db.query(Item).filter(Item.activo == True).all()
    nuevas = 0
    for item in items:
        for regla in REGLAS:
            if regla["condicion"](item):
                existe = db.query(Alert).filter(
                    Alert.item_id == item.id,
                    Alert.tipo == regla["tipo"],
                    Alert.estado.in_(["activa", "reconocida"])
                ).first()
                if not existe:
                    alerta = Alert(
                        tipo=regla["tipo"],
                        prioridad=regla["prioridad"],
                        titulo=regla["titulo"].format(nombre=item.nombre),
                        item_id=item.id,
                        item_nombre=item.nombre,
                        valor_actual=item.cantidad,
                        valor_umbral=item.stock_minimo,
                    )
                    db.add(alerta)
                    nuevas += 1
    db.commit()
```

```
db.close()
return nuevas
```

4. Endpoints REST (FastAPI)

[!TIP] Todos los endpoints viven bajo el prefijo `/alerts` y retornan respuestas en formato JSON estandar.

Ver endpoints del router

```
# GET
↳ /alerts?estado=activa&prioridad=critica&tipo=stock_bajo&page=1&limit=20
@router.get("/alerts", response_model=AlertListResponse)
async def listar_alertas(
    estado: Optional[str] = None,
    prioridad: Optional[str] = None,
    tipo: Optional[str] = None,
    page: int = 1,
    limit: int = 20,
    db: Session = Depends(get_db)
):
    ...

# PATCH /alerts/{id}/estado
@router.patch("/alerts/{id}/estado")
async def cambiar_estado(id: int, body: AlertEstadoUpdate, db: Session =
    Depends(get_db)):
    ↳ Actualiza a 'reconocida', 'resuelta', 'ignorada'
    ...

# GET /alerts/resumen
@router.get("/alerts/resumen")
async def resumen_alertas(db: Session = Depends(get_db)):
    ↳ Devuelve conteos por prioridad y tipo para las metricas del dashboard
    ...

# POST /alerts/evaluar
@router.post("/alerts/evaluar")
async def forzar_evaluacion():
    ↳ Dispara evaluar_alertas() manualmente
```

```
nuevas = evaluar_alertas()
return {"nuevas_alertas": nuevas}
```

5. Panel de alertas en Flet (frontend)

5.1 Vista principal

Ver código del dashboard

```
import flet as ft
from app.services.alert_service import AlertService

class AlertDashboard(ft.View):
    def __init__(self, page: ft.Page):
        super().__init__()
        self.page = page
        self.service = AlertService()
        self.filtro_prioridad = "todas"

    def build(self):
        resumen = self.service.get_resumen()

        # Tarjetas de metricas
        metricas = ft.Row([
            self._metric_card("Activas", resumen["total"],
                ↪ ft.colors.RED_400),
            self._metric_card("Criticas", resumen["critica"],
                ↪ ft.colors.RED_700),
            self._metric_card("Altas", resumen["alta"],
                ↪ ft.colors.ORANGE_400),
            self._metric_card("Resueltas hoy", resumen["resueltas_hoy"],
                ↪ ft.colors.GREEN_400),
        ], spacing=12)

        # Filtros
        filtros = ft.Row([
            ft.ElevatedButton("Todas", on_click=lambda e:
                ↪ self._filtrar("todas")),
            ft.ElevatedButton("Criticas", on_click=lambda e:
                ↪ self._filtrar("critica")),
```

```

        ft.ElevatedButton("Altas", on_click=lambda e:
↪ self._filtrar("alta")),
        ft.ElevatedButton("Medias", on_click=lambda e:
↪ self._filtrar("media")),
    ])

    # Tabla de alertas
    tabla = self._build_tabla()

    return ft.Column([metricas, filtros, tabla], spacing=16,
↪ expand=True)

def _metric_card(self, label, valor, color):
    return ft.Container(
        content=ft.Column([
            ft.Text(label, size=12, color=ft.colors.GREY_600),
            ft.Text(str(valor), size=24, weight=ft.FontWeight.W_500,
↪ color=color),
        ], spacing=4),
        padding=16,
        bgcolor=ft.colors.SURFACE_VARIANT,
        border_radius=8,
        expand=True,
    )

def _build_tabla(self):
    alertas = self.service.get_alertas(prioridad=self.filtro_prioridad)
    colores_prio = {
        "critica": ft.colors.RED_100,
        "alta": ft.colors.ORANGE_100,
        "media": ft.colors.BLUE_100,
        "baja": ft.colors.GREEN_100,
    }
    filas = [
        ft.DataRow(cells=[
            ft.DataCell(ft.Text(a.titulo, size=13)),
            ft.DataCell(ft.Container(
                ft.Text(a.prioridad.upper(), size=11,
↪ weight=ft.FontWeight.BOLD),
                bgcolor=colores_prio[a.prioridad], padding=4,
↪ border_radius=8)),
            ft.DataCell(ft.Text(a.estado)),
            ft.DataCell(ft.Text(a.tipo.replace("_", " ").title())),

```

```
        ft.DataCell(ft.IconButton(
            ft.icons.CHECK_CIRCLE_OUTLINE,
            on_click=lambda e, id=a.id: self._resolver(id))),
    ])
    for a in alertas
]
return ft.DataTable(
    columns=[
        ft.DataColumn(ft.Text("Alerta")),
        ft.DataColumn(ft.Text("Prioridad")),
        ft.DataColumn(ft.Text("Estado")),
        ft.DataColumn(ft.Text("Tipo")),
        ft.DataColumn(ft.Text("Accion")),
    ],
    rows=filas,
    expand=True,
)
```

5.2 Badge en el sidebar

Ver codigo del badge

```
# En el sidebar, mostrar conteo de alertas activas
alerta_badge = ft.Stack([
    ft.Icon(ft.icons.NOTIFICATIONS_OUTLINED),
    ft.Container(
        ft.Text(str(total_alertas_criticas), size=10,
↪ color=ft.colors.WHITE),
        bgcolor=ft.colors.RED,
        border_radius=99,
        padding=ft.padding.symmetric(horizontal=4, vertical=1),
        right=0, top=0,
    )
])
```

5.3 Toast de nueva alerta

Ver codigo del toast

```
def mostrar_toast_alerta(page, mensaje, prioridad="alta"):
    colores = {"critica": ft.colors.RED_400, "alta": ft.colors.ORANGE_400,
↪ "media": ft.colors.BLUE_400}
```

```

snack = ft.SnackBar(
    content=ft.Row([
        ft.Icon(ft.icons.WARNING_AMBER_ROUNDED, color=ft.colors.WHITE),
        ft.Text(mensaje, color=ft.colors.WHITE),
    ]),
    bgcolor=colores.get(prioridad, ft.colors.GREY_700),
    duration=5000,
    action="Ver",
    on_action=lambda e: page.go("/alertas"),
)
page.overlay.append(snack)
snack.open = True
page.update()

```

6. Scheduler automatico de evaluacion

[!IMPORTANT] El scheduler ejecuta `evaluar_alertas()` cada **15 minutos**. Se integra con el ciclo de vida de FastAPI mediante el patron `lifespan`.

Ver configuracion del scheduler

```

# En main.py o en un worker aparte con APScheduler
from apscheduler.schedulers.background import BackgroundScheduler
from app.alerts.rules import evaluar_alertas

scheduler = BackgroundScheduler()
scheduler.add_job(evaluar_alertas, "interval", minutes=15,
    ↪ id="motor_alertas")
scheduler.start()

```

Opcionalmente, con FastAPI `lifespan`:

Ver integracion con `lifespan`

```

from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    scheduler.start()
    yield
    scheduler.shutdown()

```

```
app = FastAPI(lifespan=lifespan)
```

7. Tipos de alertas recomendados

Tipo	Disparador	Prioridad	Frecuencia
Stock minimo	cantidad < stock_minimo	Critica	Cada 15 min
Agotado	cantidad == 0	Critica	Cada 15 min
Proximo a vencer	fecha_vencimiento <= 30 dias	Alta	Diaria
Vencido	fecha_vencimiento < hoy	Critica	Diaria
Sin movimiento	ultima transaccion > 90 dias	Media	Semanal
Sobrestock	cantidad > stock_maximo	Baja	Diaria
Discrepancia	diferencia fisico vs sistema > 5%	Alta	Tras conteo
Proveedor tardio	orden compra vencida sin recibir	Alta	Diaria
Valor alto estancado	item costoso sin rotacion	Media	Semanal

8. Vistas recomendadas

8.1 Dashboard (vista principal)

[!TIP] Agrupa la informacion mas critica en la parte superior para que el usuario pueda tomar accion inmediata.

- **4 tarjetas metricas:** total activas, criticas, tiempo prom. resolucion, resueltas hoy
- **Grafica de barras:** alertas por categoria
- **Grafica de linea:** tendencia de alertas por dia (ultimos 7 dias)
- **Tabla de cola** con filtros por prioridad y estado

8.2 Vista de detalle de alerta

- Encabezado con titulo, badge de prioridad y estado
- Linea de tiempo de cambios de estado
- Campo de notas / comentarios
- Botones: **Reconocer**, **Resolver**, **Asignar**, **Ignorar**

8.3 Configuracion de reglas

- Lista de reglas con toggle activar/desactivar
- Formulario para editar umbrales (stock minimo, dias de anticipacion, etc.)
- Historial de disparos por regla

8.4 Historial

- Tabla de alertas resueltas e ignoradas
- Filtros por rango de fechas, tipo, item
- Exportacion a Excel

9. Mejoras adicionales recomendadas

9.1 Agrupacion de alertas

[!TIP] Evita ruido visual agrupando alertas del mismo tipo en una sola tarjeta resumen.

*# En lugar de 50 alertas de stock bajo separadas:
-> "15 items con stock bajo en categoria EPP"*

9.2 Silenciar / posponer alertas (snooze)

```
ALTER TABLE alerts ADD COLUMN snoozed_until DATETIME NULL;
```

[!NOTE] El motor salta alertas con `snoozed_until > NOW()` para respetar el periodo de snooze.

9.3 Asignacion de alertas

```
# Asignar alerta a un usuario responsable
PATCH /alerts/{id}/asignar
{"asignado_a": user_id}
```

9.4 Notificaciones push / WhatsApp

[!WARNING] Las integraciones externas requieren claves de API. Guardalas en **variables de entorno** o en un vault de secretos.

```
import httpx

async def notificar_whatsapp(numero, mensaje):
    # Integracion con WhatsApp Business API o Twilio
    async with httpx.AsyncClient() as client:
        await client.post(WHATSAPP_URL, json={"to": numero, "message":
            ↪ mensaje})
```

9.5 SLA y escalamiento automatico

[!IMPORTANT] Si una alerta **critica** lleva mas de **2 horas** sin reconocer -> escalar al supervisor automaticamente.

```
def escalar_alertas_vencidas():
    alertas = db.query(Alert).filter(
        Alert.prioridad == "critica",
        Alert.estado == "activa",
        Alert.created_at < datetime.now() - timedelta(hours=2)
    ).all()
    for a in alertas:
        notificar_supervisor(a)
```

9.6 KPIs del modulo de alertas

[!TIP] Calcula y muestra estos KPIs en el dashboard para medir la *salud* del proceso de alertas.

KPI	Descripcion
MTTR	<i>Mean Time To Resolve</i> – promedio de minutos desde creacion hasta resolucion
Tasa de reconocimiento	% de alertas reconocidas en < 30 min
Top items problematicos	Items con mas alertas historicas
Eficacia de reglas	Cuales reglas generan mas alertas validas vs ignoradas

9.7 Deduplicacion inteligente

[!NOTE] Antes de crear una alerta, se verifica el **cooldown** configurado en la regla para evitar duplicados.

```
ultima = db.query(Alert).filter(  
    Alert.item_id == item.id,  
    Alert.tipo == tipo,  
    Alert.created_at > datetime.now() - timedelta(hours=cooldown_horas)  
)<div data-bbox="113 662 467 678" data-label="Section-Header">

## 9.8 Exportacion de reporte de alertas


```

Ver generador de Excel

```
import openpyxl<div data-bbox="113 899 314 916" data-label="Page-Footer">

---

Securitas Colombia S.A.
```

```
for a in alertas:
    ws.append([a.id, a.tipo, a.prioridad, a.titulo, a.item_nombre,
               a.estado, str(a.created_at), str(a.resolved_at or "")])
wb.save(ruta_salida)
```

10. Checklist de implementacion

Base

- ☒ Crear tabla `alerts` y `alert_rules` en MySQL
- ☐ Implementar modelo SQLAlchemy `Alert`
- ☐ Implementar motor de reglas `evaluar_alertas()`
- ☐ Conectar scheduler (APScheduler) cada 15 minutos

API REST

- ☐ Endpoints REST: GET, PATCH estado, GET resumen

Frontend Flet

- ☐ Vista Flet: dashboard con metricas + tabla + filtros
- ☐ Badge de alertas en el sidebar
- ☐ Toast de nueva alerta critica

Detalle y configuracion

- ☐ Vista de detalle con linea de tiempo
- ☐ Configuracion de reglas (toggle + umbrales)
- ☐ Historial y exportacion Excel

Mejoras

- ☐ Logica de cooldown (deduplicacion)
- ☐ Logica de escalamiento por SLA
- ☐ (Opcional) Integracion WhatsApp para criticas

*Documento generado para **PROYECTO_SECURITAS – Inventario_SE** - Compatible con FastAPI + Flet + MySQL + APScheduler*